

CONTRATO DE PRESTAÇÃO DE SERVIÇOS

Contrato de Engenharia de Software & Especificação Técnica de API RESTful

CONTRATANTE: TechLib Solutions S.A., pessoa jurídica de direito privado, inscrita no CNPJ sob o nº 99.999.999/0001-99, com sede na Cidade de Tecnologia, doravante denominada simplesmente CONTRATANTE.

CONTRATADO: O desenvolvedor discente do Curso de Backend com TypeScript, doravante denominado simplesmente CONTRATADO.

As partes acima qualificadas resolvem firmar o presente instrumento para o desenvolvimento do ecossistema backend do Sistema de Gestão de Acervo e Empréstimos Bibliográficos, mediante as cláusulas e condições técnicas descritas a seguir.

CLÁUSULA PRIMEIRA: DO OBJETO E STACK TECNOLÓGICA

O presente contrato tem por objeto a prestação de serviços de desenvolvimento de uma API RESTful completa, utilizando estritamente a stack de tecnologias homologada pela arquitetura interna da CONTRATANTE, descrita no escopo do projeto:

- **Ambiente de Execução e Linguagem:** Node.js estruturado com a linguagem TypeScript para tipagem estática e segurança em tempo de compilação.
- **Framework Web principal:** Express para roteamento e gerenciamento do ciclo de requisição e resposta.
- **Camada de Persistência (ORM):** Prisma ORM atuando como ponte de dados e gerenciador de migrations.
- **Padrão de Arquitetura:** Repository Pattern, implementando a abstração especializada através da estrutura VSRepository.
- **Validação de Dados:** Biblioteca Zod para parsing de schemas e validação rigorosa dos payloads de entrada (body, query, params).
- **Autenticação e Segurança:** Implementação de JsonWebToken (JWT) para sessões assíncronas e stateless, além de integração com Login Social (Google Auth).
- **Camada de Performance e Cache:** Utilização do Redis (via cliente ioredis) para otimização de consultas pesadas e cache estratégico.
- **Resiliência e Proteção de Infraestrutura:** Mecanismo de Rate Limit (via express-rate-limit) para prevenção de ataques de negação de serviço (DoS/DDoS) e abuso de endpoints.
- **Banco de Dados Relacional:** PostgreSQL.

CLÁUSULA SEGUNDA: ESPECIFICAÇÃO DO MODELO DE DADOS (DICIONÁRIO DE DADOS)

O CONTRATADO obriga-se a projetar e implementar o banco de dados relacional realizando a normalização e o mapeamento das entidades exigidas pela arquitetura de dados da CONTRATANTE. O sistema deverá garantir a persistência das seguintes estruturas abstratas e suas respectivas regras de negócio:

1. Entidade: Autores

Responsável por armazenar o catálogo de escritores do acervo bibliográfico.

- **Identificador:** UUID (Chave Primária gerada automaticamente por funções nativas).
- **Atributos Obrigatórios:** Nome (limite de 150 caracteres), Data de Nascimento e Descrição (biografia curta com limite de 300 caracteres).
- **Controle de Estado:** Indicador lógico de atividade (booleano) iniciando como verdadeiro, de modo a viabilizar o mecanismo de Soft Delete.
- **Auditoria:** Registro automático de Data/Hora de Criação e de Última Atualização.

2. Entidade: Livros

Responsável por manter os dados estruturais da obra e o seu controle rígido de estoque.

- **Identificador:** UUID (Chave Primária gerada automaticamente).
- **Atributos Obrigatórios:** ISBN (Código internacional único do livro, com limite de 20 caracteres), Título (limite de 150 caracteres), Sinopse detalhada (limite de 1000 caracteres), Data de Publicação e Preço (formato decimal monetário de alta precisão).
- **Controle de Inventário:** Quantidade em Estoque (total geral registrado) e Quantidade Disponível (variável deduzida dinamicamente conforme empréstimos ativos).
- **Relacionamento:** Vínculo referencial obrigatório com o Autor da obra (Chave Estrangeira).
- **Controle e Auditoria:** Indicador lógico de atividade (booleano) e registros automáticos de Data/Hora de Criação e Atualização.

3. Entidades: Gêneros e Categorização (Relacionamento N:M)

Considerando que um livro pode pertencer a múltiplos gêneros e um gênero engloba múltiplos livros, a modelagem exige uma estrutura associativa explícita:

- **Tabela Principal de Gêneros:** Contém UUID (Chave Primária), Nome do Gênero (obrigatório, único, limite de 30 caracteres) e registro de Data/Hora de Criação.
- **Tabela Associativa (Livro_Genero):** Armazena o vínculo direto entre o Livro e o Gênero por meio de suas respectivas Chaves Estrangeiras. Deve possuir obrigatoriamente uma trava de unicidade composta (Unique Constraint) para impedir a duplicidade de uma mesma categorização.

4. Entidade: Usuários e Sistema de Cargos

Responsável pelo controle de acesso, identificação e autenticação dos agentes no sistema.

- **Identificador:** UUID (Chave Primária gerada automaticamente).
- **Atributos Obrigatórios:** Nome (limite de 150 caracteres), E-mail (único no sistema, limite de 254 caracteres) e Senha (armazenada obrigatoriamente sob hash criptográfico seguro).

- **Regra de Acesso (RBAC):** Implementação baseada em tipo enumerado (Enum) nativo, composto estritamente pelas permissões: **CLIENTE**, **OPERADOR** e **ADMIN**.
- **Auditoria:** Registro cronológico automático de Data/Hora de Criação e Atualização.

5. Entidade: Controle de Empréstimos

Responsável por registrar e auditar as transações de locação de livros do acervo.

- **Identificador:** UUID (Chave Primária gerada automaticamente).
- **Relacionamentos:** Vínculos obrigatórios (Chaves Estrangeiras) indicando de forma explícita qual Livro foi locado e qual Usuário realizou a operação.
- **Controle de Prazos:** Data de Devolução Prevista (definida no ato do empréstimo) e Data de Devolução Real (admitindo valor nulo até a entrega efetiva do exemplar).
- **Auditoria:** Registro automático de Data/Hora de Criação da transação.

CLÁUSULA TERCEIRA: CONTROLE DE ACESSO BASEADO EM REGRAS (RBAC)

A API deverá, obrigatoriamente, validar o cargo (cargo_usuario) contido no payload decodificado do token JWT gerado no login. Fica estipulada a seguinte matriz rígida de controle de acesso aos recursos:

RECURSO / OPERAÇÃO	MAPEAMENTO HTTP / ENDPOINT	NÍVEL DE PERMISSÃO MÍNIMO EXIGIDO
Adicionar Gênero	POST /generos	Apenas ADMIN
Remover Gênero	DELETE /generos/:id	Apenas ADMIN
Remover Autor (Soft Delete)	DELETE /autores/:id	Apenas ADMIN
Remover Livro (Soft Delete)	DELETE /livros/:id	Apenas ADMIN
Visualizar Registros Removidos (Inativos)	GET /admin/auditoria/... (ou filtros)	Apenas ADMIN
Criar Autor / Livro	POST /autores ou POST /livros	Permitido para OPERADOR e ADMIN
Editar Autor / Livro	PUT /autores/:id ou PUT /livros/:id	Permitido para OPERADOR e ADMIN
Solicitar Empréstimo de Livro	POST /emprestimos	Apenas CLIENTE
Listagem Geral de Livros Ativos	GET /livros	Público / Todos os cargos autenticados

Nota de Segurança Importante: Qualquer requisição efetuada a um endpoint sem o nível de permissão adequado deve retornar imediatamente o código de status HTTP **403 Forbidden**, registrando a tentativa indevida nos logs da aplicação.

CLÁUSULA QUARTA: DOS REQUISITOS ESPECIAIS DE IMPLEMENTAÇÃO

1. Mecanismo de Soft Delete (Exclusão Lógica)

Fica terminantemente proibida a execução de comandos SQL do tipo DELETE físico nas tabelas livro e autor. O CONTRATADO deverá implementar a exclusão por meio de Soft Delete:

- Ao acionar as rotas de remoção (DELETE /livros/:id e DELETE /autores/:id), a API deverá atualizar o campo lógico ativo para false e registrar o momento exato na coluna data_atualizacao.
- **Filtro Base de Consulta:** Todas as rotas tradicionais de listagem e busca pública (ex: consulta de catálogo para Clientes e Operadores) devem conter a cláusula padrão de filtragem para exibir apenas registros onde ativo = true.
- **Exceção de Visualização:** Somente usuários autenticados com o cargo de ADMIN terão permissão de passar parâmetros adicionais ou acessar endpoints específicos para visualizar os livros e autores que foram movidos para o estado inativo (removidos logicamente).

2. Validação e Tipagem

Todos os payloads de entrada devem passar por um pipeline de validação estruturado com esquemas do Zod antes de alcançarem a camada do controller ou repository. Dados inválidos devem retornar status HTTP **400 Bad Request** contendo os detalhes específicos dos campos violados.

3. Desempenho e Cache

As listagens de livros com alta recorrência de busca devem ser cacheadas utilizando o Redis. O cache deve ser invalidado automaticamente sempre que houver inserção, alteração de dados (PUT) ou Soft Delete (DELETE) dos recursos afetados.

CLÁUSULA QUINTA: DO CRONOGRAMA E FASES DE ENTREGA ESTIMADAS

O desenvolvimento do projeto seguirá o cronograma técnico de estimativa continuada estruturado em 4 fases principais (aulas de entrega):

- **Fase 1: Inicialização do ecossistema e configuração do ambiente corporativo.** Instalação das dependências estruturais do Node.js com TypeScript e inicialização do servidor HTTP base utilizando Express.
- **Fase 2: Provisionamento do Banco de Dados PostgreSQL via contêineres Docker.** Criação das migrations e geração do Client do Prisma ORM baseado no modelo de dados acordado. Implementação das rotas iniciais de cadastro.
- **Fase 3: Finalização dos fluxos principais de CRUD e proteção.** Acoplamento das camadas de validação via Zod. Desenvolvimento do sistema central de autenticação e proteção de rotas através da emissão e validação de tokens JWT associados aos cargos (RBAC).
- **Fase 4: Refinamento e robustez da infraestrutura.** Implementação do fluxo de autenticação por login social com o Google. Integração da camada de cache distribuído via Redis e blindagem das rotas públicas e privadas com Rate Limit (express-rate-limit).

Por estarem em perfeito acordo com todas as diretrizes técnicas de Engenharia de Software descritas neste documento de requisitos e contrato de simulação corporativa, as partes dão por aprovada a arquitetura técnica.

TechLib Solutions S.A.
Diretoria de Engenharia de Software

CONTRATADO (Engenheiro de Software)
Aluno do Curso de Backend Node.js + TypeScript